COMPUTER VISION

LABSHEET-6

NAME-HARSH RAJ

CU ID -240251762

#2. Required Libraries

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import data
from sklearn.cluster import KMeans
```

#3.Load and Preprocess a Real-World Image

```
# Load real-world image
image = data.astronaut()

# Convert RGB to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

# Reduce noise using Gaussian Blur
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(image)
```

```
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(blurred, cmap="gray")
plt.title("Grayscale + Gaussian Blur")
plt.axis("off")

plt.show()

#4. Global Thresholding
# Apply global thresholding
threshold_value = 127

_, global_thresh = cv2.threshold(
    blurred, threshold_value, 255, cv2.THRESH_BINARY
)

plt.imshow(global_thresh, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")
plt.show()

#5. Otsu’s Thresholding
# Apply Otsu's thresholding
otsu_value, otsu_thresh = cv2.threshold(
```

```
    blurred, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

print("Optimal Otsu Threshold:", otsu_value)

plt.imshow(otsu_thresh, cmap="gray")
plt.title("Otsu's Thresholding")
plt.axis("off")
plt.show()

#6. Adaptive Thresholding
adaptive_thresh = cv2.adaptiveThreshold(
    blurred,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

plt.imshow(adaptive_thresh, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")
plt.show()

#7. Watershed Segmentation
```

```
# Use Otsu's result as a binary image
binary = otsu_thresh

# Remove noise using morphological opening
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel, iterations=2)

# Find sure background
sure_bg = cv2.dilate(opening, kernel, iterations=3)

# Find sure foreground using distance transform
dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)

_, sure_fg = cv2.threshold(
    dist_transform, 0.5 * dist_transform.max(), 255, 0
)

sure_fg = np.uint8(sure_fg)

# Unknown region
unknown = cv2.subtract(sure_bg, sure_fg)

# Marker labelling
num_markers, markers = cv2.connectedComponents(sure_fg)

markers = markers + 1
```

```
markers[unknown == 255] = 0

# Apply watershed
image_bgr = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
markers = cv2.watershed(image_bgr, markers)

# Mark boundaries in red
result = image.copy()
result[markers == -1] = [255, 0, 0]

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(sure_fg, cmap="gray")
plt.title("Sure Foreground")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(result)
plt.title("Watershed Segmentation")
plt.axis("off")

plt.show()

#8. K-Means Color Segmentation
# Reshape image into a list of pixels
```

```
pixels = image.reshape((-1, 3))

# Convert to float32
pixels = np.float32(pixels)

# Define criteria
criteria = (
    cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
    100,
    0.2
)

# Number of clusters
K = 3

# Apply K-Means clustering
_, labels, centers = cv2.kmeans(
    pixels,
    K,
    None,
    criteria,
    10,
    cv2.KMEANS_RANDOM_CENTERS
)

# Convert centers to uint8
```

```
centers = np.uint8(centers)

# Reconstruct segmented image
segmented = centers[labels.flatten()]
segmented = segmented.reshape(image.shape)

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(segmented)
plt.title("K-Means Color Segmentation")
plt.axis("off")

plt.show()

#10. Visualize All Results Together
plt.figure(figsize=(15, 8))

plt.subplot(2, 3, 1)
plt.imshow(image)
plt.title("Original")
```

```
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(global_thresh, cmap="gray")
plt.title("Global Threshold")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(otsu_thresh, cmap="gray")
plt.title("Otsu")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(adaptive_thresh, cmap="gray")
plt.title("Adaptive")
plt.axis("off")

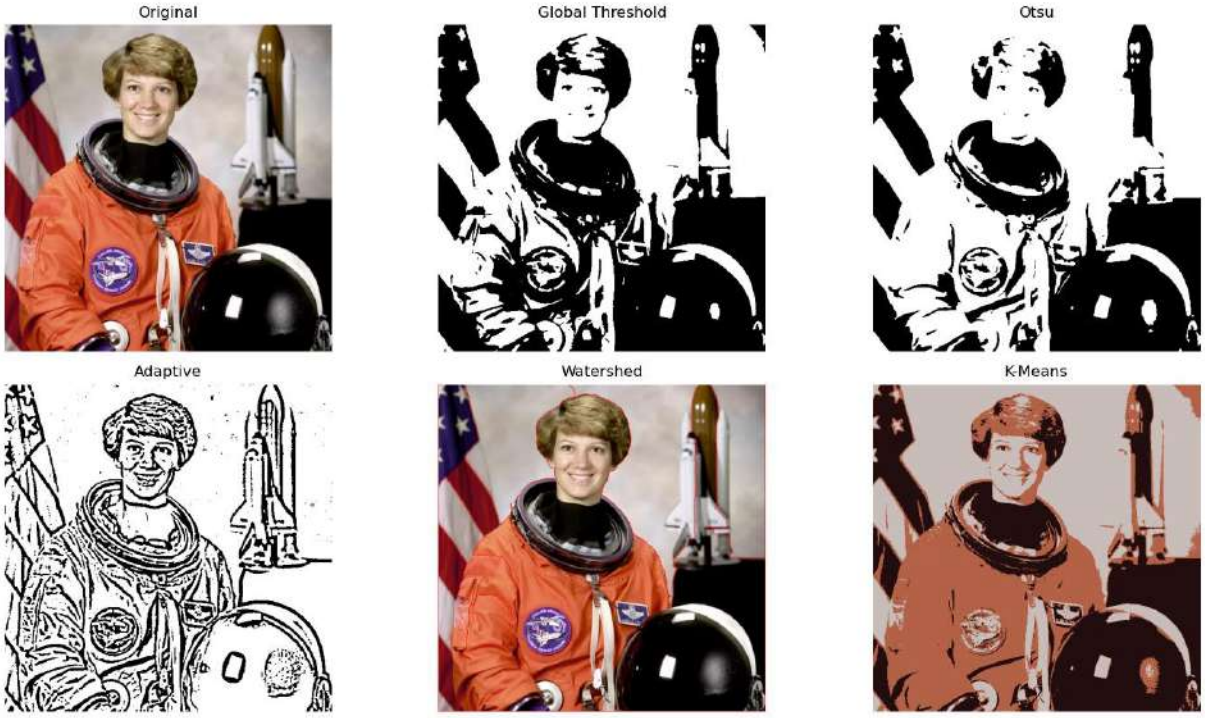
plt.subplot(2, 3, 5)
plt.imshow(result)
plt.title("Watershed")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(segmented)
plt.title("K-Means")
plt.axis("off")
```

```
plt.tight_layout()

plt.show()
```

OUTPUT:-



QUESTION AND ANSWER

1. What is image segmentation? Why is it considered a fundamental step in computer vision?

Image segmentation is the process of dividing an image into meaningful regions or objects. It is fundamental because it helps identify and separate objects for further analysis.

2. Differentiate between Image Segmentation and Image Classification.

Image classification assigns a label to an entire image, while image segmentation assigns labels to individual pixels or regions. For example, classification identifies a cat, while segmentation identifies the exact pixels belonging to the cat.

3. Explain the working principle of Global Thresholding, Otsu's Thresholding, and Adaptive Thresholding.

Global Thresholding: Uses a fixed threshold to separate foreground and background. Otsu's Thresholding: Automatically selects the optimal threshold by minimizing within-class intensity variance. Adaptive Thresholding: Calculates different thresholds for local regions, making it suitable for uneven illumination.

4. What is the Watershed Algorithm? Why is it useful for separating overlapping objects?

Watershed treats an image like a topographic surface and separates regions using boundaries called watershed lines. It is useful for dividing touching or overlapping objects into individual regions.

5. How is K-Means Clustering applied to image segmentation?

K-Means groups pixels into K clusters based on similarities such as color or intensity. Each pixel is assigned to a cluster, and the clusters form the segmented image.

6. Compare threshold-based segmentation and clustering-based segmentation techniques.

Threshold-based segmentation uses intensity values to separate regions and is simple and fast. Clustering-based segmentation groups pixels based on similarity and can handle multiple regions, but is generally more computationally expensive.

7. What challenges are encountered while segmenting images with complex backgrounds or varying illumination?

Challenges include noise, shadows, low contrast, uneven lighting, overlapping objects, and similar foreground-background colors. These conditions make it difficult to identify accurate object boundaries.

8. Mention five real-world applications where image segmentation plays a critical role.

Medical image analysis ,Satellite image interpretation ,Autonomous vehicles ,Face and biometric recognition ,Industrial quality inspection

9. How does image segmentation improve the performance of object detection and image recognition systems?

Segmentation separates objects from the background and provides accurate object boundaries. This helps systems focus on relevant regions, improving recognition accuracy and detection performance.

10. Compare traditional image segmentation techniques with deep learning-based segmentation methods such as U-Net and Mask R-CNN.

Traditional techniques such as thresholding and K-Means are simple, fast, and require less training data, but may struggle with complex images. Deep learning methods such as U-Net and Mask R-CNN learn features automatically and provide more accurate segmentation, but require large datasets, training time, and greater computational resources.